

CAHIER DES CHARGES

Boutique en ligne - Items Rocket League



Développeur Web Full-Stack
RNCP 37273

Tommy KAROTSCH
Bachelor 1ère année Cannes

Sommaire

1. Présentation du projet	3
1.1 Présentation générale	
1.2 Public cible	
1.3 Enjeux du projet	
2. Fonctionnalités	5
2.1 Utilisateur	
2.2 Administrateur	
3. Gestion de projet	6
3.1 Planning de projet	
3.2 Outils utilisés	
4. Identité visuelle & UX/UI	7
4.1 Charte graphique	
4.2 Wireframe	
4.3 Maquettes	
4.4 Accessibilité RGAA	
5. Conception de la base de données	10
5.1 MCD - Modèle Conceptuel de Données	
5.2 MLD - Modèle Logique de Données	
5.3 MPD - Modèle Physique de Données	
6. Développement Front-end	13
6.1 Responsive	
6.2 Sass	
6.3 Fetch api	
7. Développement Back-end	16
7.1 Stack technique	
7.2 Architecture MVC	
7.3 API REST	
7.4 POO / PDO	
8. Sécurité	17
8.1 Authentification & hachage	
8.2 Protections XSS / Injections SQL	
8.3 Conformité RGPD	
9. Cadre pédagogique	18
9.1 Blocs de compétences - RNCP 37273	
10. Annexe	19
10.1 Header & Hero Section	
10.2 Main Section	
10.3 Footer	

1. Présentation du projet

1.1 Présentation générale

Objectif du projet : Développer une boutique en ligne inspirée des mécaniques de *Rocket League*. La plateforme doit transposer l'économie du jeu vidéo dans le e-commerce réel à travers deux fonctionnalités clés : une classification des produits par niveaux de rareté (avec gestion de variantes et couleurs précises).

Concrètement, l'idée est de développer un site e-commerce complet dédié à la vente fictive d'items Rocket League. Le catalogue couvrira toutes les grandes catégories d'objets du jeu :

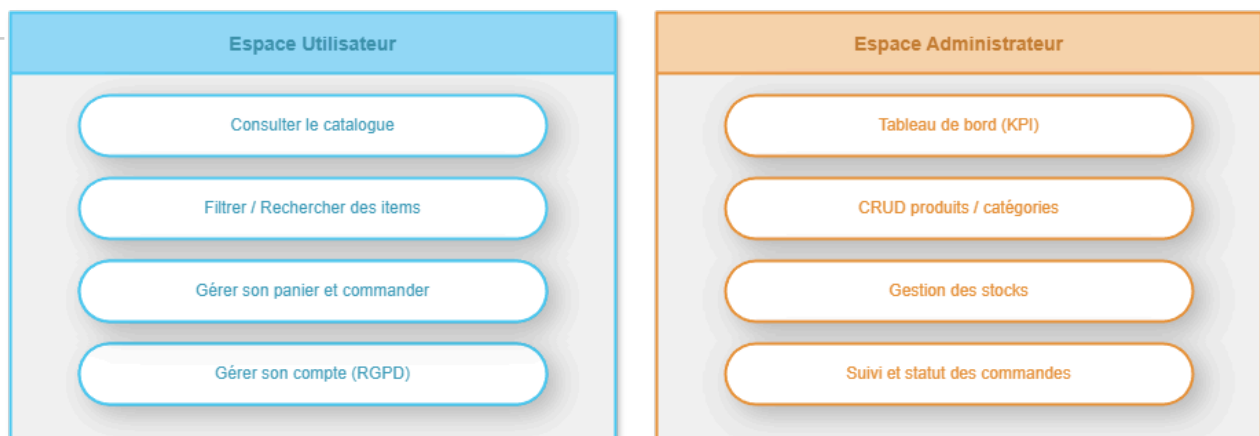
- les voitures (Bodies), les stickers (Decals)
- les roues (Wheels), les boosts
- les traînées (Trails)
- les explosions de but (Goal Explosions)
- les chapeaux (Toppers) et les antennes (Antennas).

Chaque item aura une rareté allant de Commun jusqu'au rarissime : Marché noir et une couleur spécifique parmi une palette définie.

J'ai fait le choix d'une approche mobile-first dès le départ, parce que le public cible des joueurs est entre 13 et 35 ans.

Il ne s'agit pas juste de « rendre le site responsive », mais bien de penser chaque composant d'abord pour un petit écran, puis d'agrandir l'expérience sur desktop.

Le site se découpera en deux grands espaces : une interface publique pour les utilisateurs (catalogue, panier, commande, compte) et un back-office pour l'administrateur (gestion des produits, des stocks et des commandes).



1.2 Public cible

Le site s'adresse principalement aux joueurs de Rocket League, un public qui connaît parfaitement les mécaniques de rareté et de personnalisation propres au jeu. Ce sont des gens habitués à naviguer vite, à chercher un item précis, à comparer. La tranche d'âge visée se situe entre 13 et 35 ans un public très à l'aise avec les interfaces numériques modernes, et qui consulte majoritairement depuis son téléphone.

Ce public n'a pas besoin qu'on lui explique ce qu'est un Decal Exotique ou une roue Cobalt il sait exactement ce qu'il cherche. Mon travail est de lui permettre de le trouver vite, avec une interface claire et une navigation fluide.

L'espace administrateur, lui, s'adresse à un profil très différent : un gestionnaire de boutique qui a besoin d'efficacité avant tout. Peu importe qu'il connaisse Rocket League ou non : il doit pouvoir gérer le catalogue, surveiller les stocks et suivre les commandes.

Critère	Espace Utilisateur	Espace Administrateur
Public	Joueurs de Rocket League	Gestionnaire de boutique
Tranche d'âge	13 – 35 ans	Non définie
Connaissance du jeu	Experte — connaît les Decals, raretés, roues, etc.	Non requise — peut ne pas connaître Rocket League
Priorité principale	Trouver un item précis rapidement et comparer	Efficacité dans la gestion quotidienne
Appareil principal	Mobile (majoritaire)	Desktop (probable)
Rapport à l'interface	Très à l'aise avec les interfaces numériques modernes	Besoin d'une interface claire et directe, sans friction
Objectif UX	Navigation fluide, recherche rapide, catalogue bien structuré	Gérer le catalogue, surveiller les stocks, suivre les commandes

1.3 Enjeux du projet

L'enjeu Technique : maintenir une architecture propre et cohérente sur l'ensemble du projet comme : en respectant le pattern MVC , hacher les mots de passe, utiliser des requêtes préparées, échapper les sorties HTML.

L'enjeu UX est un peu plus délicat : créer une interface qui ressemble à l'univers Rocket League sans sacrifier la lisibilité.

2. Fonctionnalités

2.1 Les fonctionnalités côté Utilisateur

- **Page d'accueil**
 - Mise en avant des items les plus rares ou les plus populaires sous forme de vignettes visuelles.
- **Catalogue**
 - Système de filtres cumulables (par type d'objet, par rareté et par couleur).
 - Options de tri (par prix et par rareté).
- **Fiche produit détaillée**
 - Affichage des informations : image, nom, type, rareté, couleur, description, prix, stock.
 - Bouton d'ajout au panier visible et accessible immédiatement.
- **Panier persistant**
 - Sauvegarde en session pour les utilisateurs non connectés.
 - Sauvegarde en base de données pour les utilisateurs connectés.
 - Modification des quantités et suppression d'articles.
 - Affichage total mis à jour en temps réel.
- **Processus de commande**
 - Déroulement en étapes claires :
 - Validation du panier.
 - Renseignement de l'adresse de livraison.
 - Récapitulatif.
 - Simulation du paiement.
- **Espace compte utilisateur**
 - Inscription et connexion.
 - Modification des informations personnelles (pseudo, email, mot de passe, adresse).
 - Consultation de l'historique des commandes.
 - Demande de suppression complète du compte et de toutes les données associées.
- **Accessibilité (Normes RGAA)**
 - Contrastes suffisants.
 - Balises sémantiques.
 - Attributs "ALT" sur toutes les images.
 - Labels associés à tous les champs.

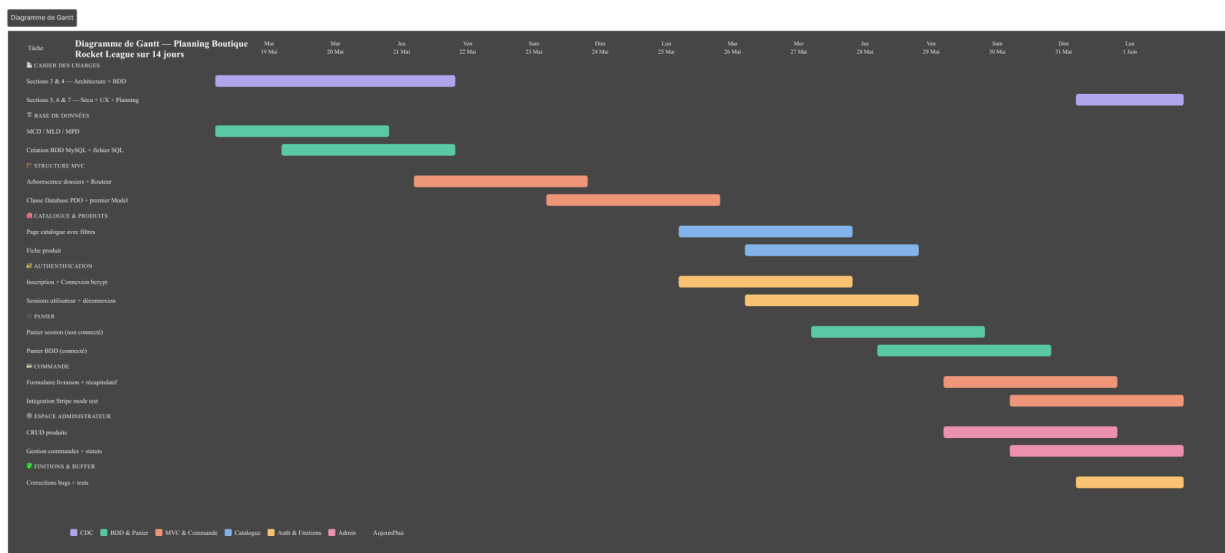
2.2 Les fonctionnalités côté Administrateur

- **Tableau de bord synthétique**
 - Affichage du nombre de commandes.
 - Affichage des produits récemment ajoutés.

- **Gestion des produits (via un formulaire simple)**
 - Création d'un item.
 - Modification de la description, du prix, du stock et de l'image.
- **Gestion de la structure du catalogue**
 - Création ou suppression des types d'objets.
 - Création ou suppression des raretés.
 - Création ou suppression des couleurs.
- **Suivi des commandes**
 - Visualisation de tous les achats effectués.
 - Filtrage des commandes par statut.
 - Gestion du cycle de vie des commandes (progression de « En attente » à « Expédiée », puis « Livrée »).

3. Gestion de projet

3.1 Planning de projet



3.2 Outils utilisés

Les outils utilisés sont :

- **VS Code** pour le développement
- **Drawio** (VS Code) pour le MCD / MLD / MPD
- **Trello** pour la gestion des tâches même si j'ai réalisé le projet seul
- **Github** pour le versionning
- **Excel** pour les différents tableau
- **Figma** pour les maquettes

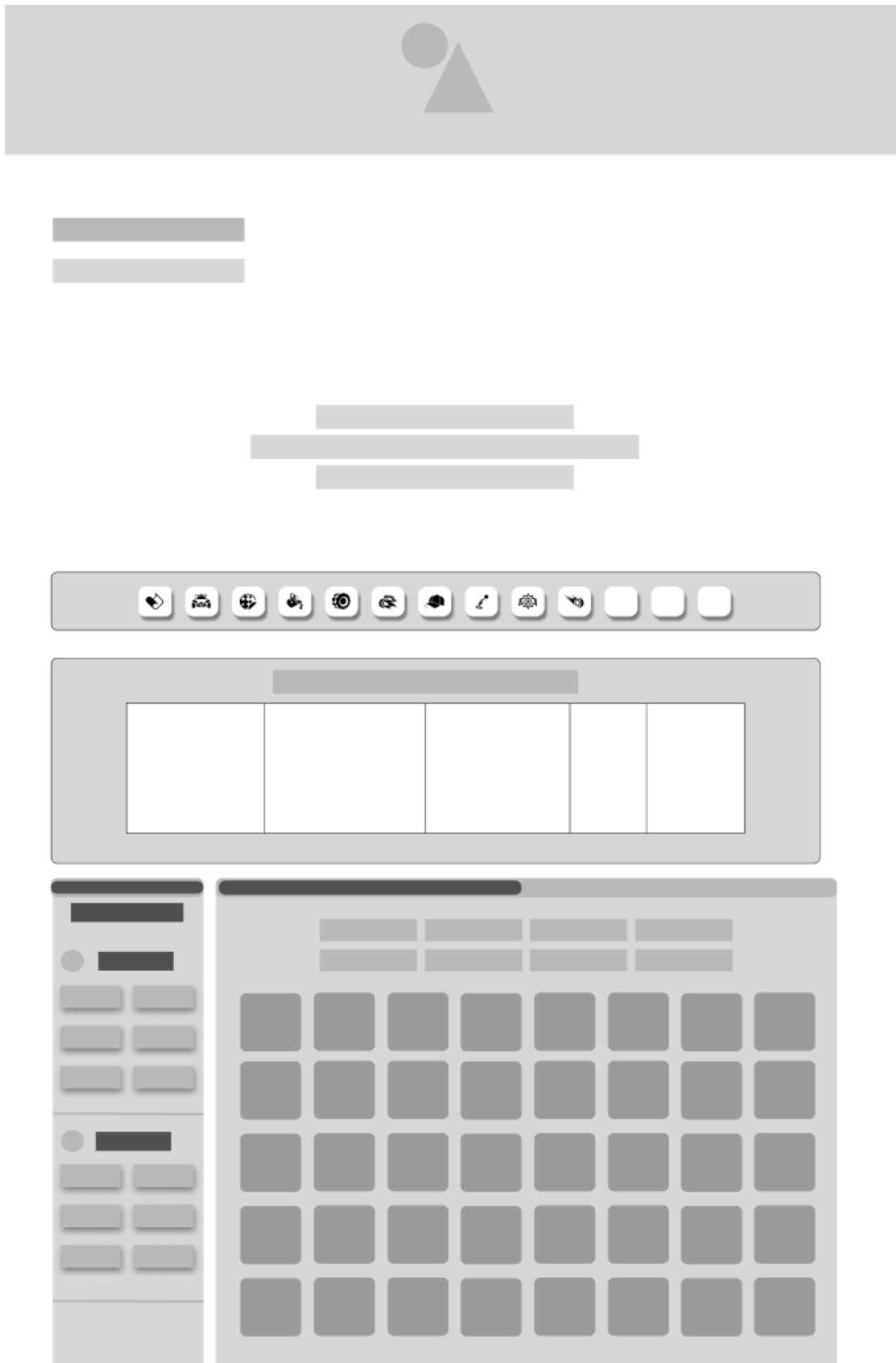
4. Identité visuelle & UX/UI

4.1 Charte graphique

L'univers visuel de Rocket League est très marqué : couleurs vives, effets lumineux, dynamisme. J'ai envie que l'interface s'en inspire, tout en restant sobre et lisible pour ne pas sacrifier l'expérience d'achat. L'idée est d'utiliser un fond sombre ou très légèrement teinté, avec des accents de couleurs vives (bleu électrique, orange, blanc) pour mettre en valeur les items.

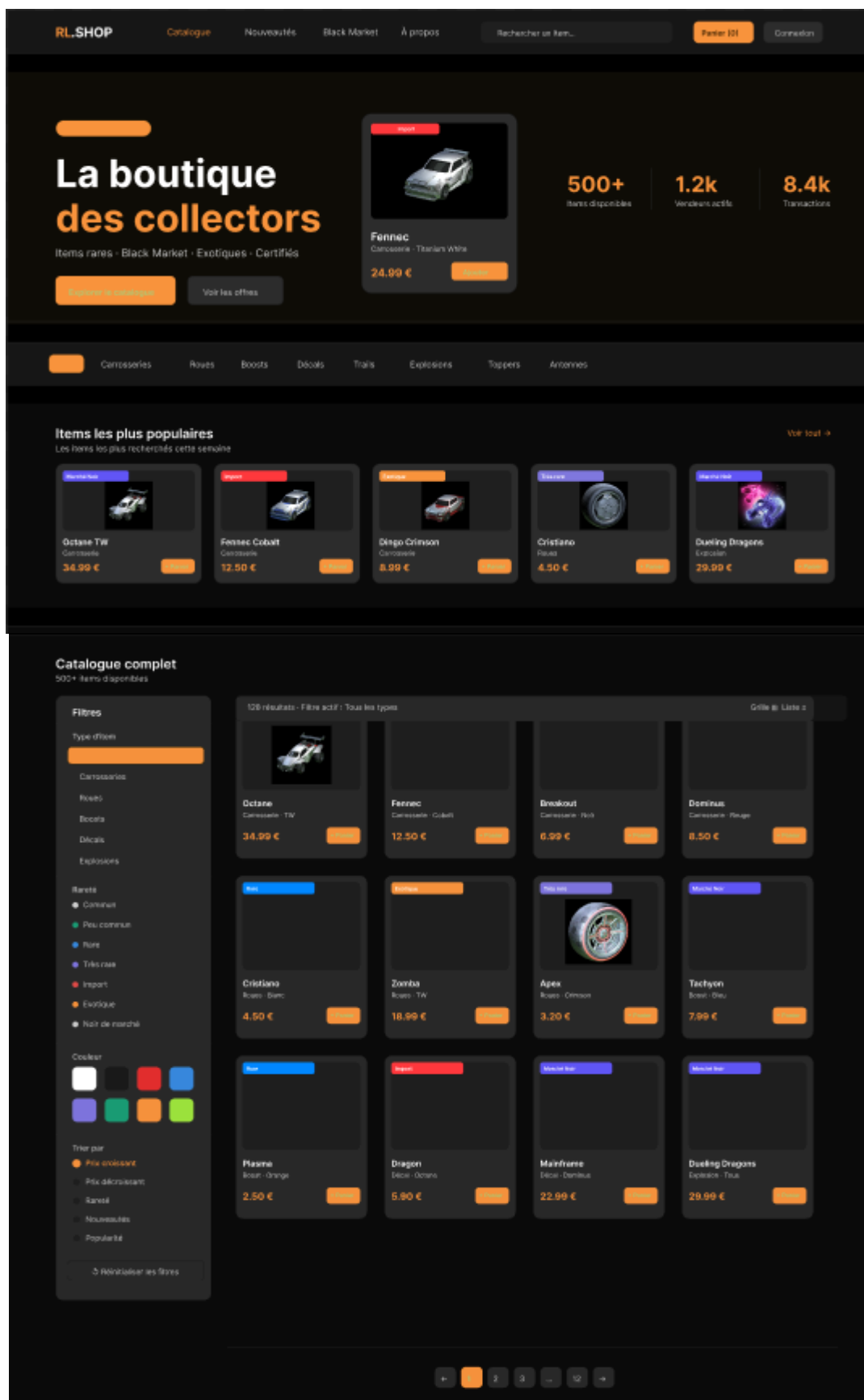


4.2 Wireframe



4.3 Maquettes

Les maquettes sont réalisées sur Figma et couvriront l'ensemble des pages clés : page d'accueil, catalogue avec filtres, fiche produit, panier, tunnel de commande (les 5 étapes), espace compte utilisateur, et back-office administrateur. Chaque maquette sera déclinée en version mobile et desktop. (Voir Annexe)



4.3 Accessibilité RGAA

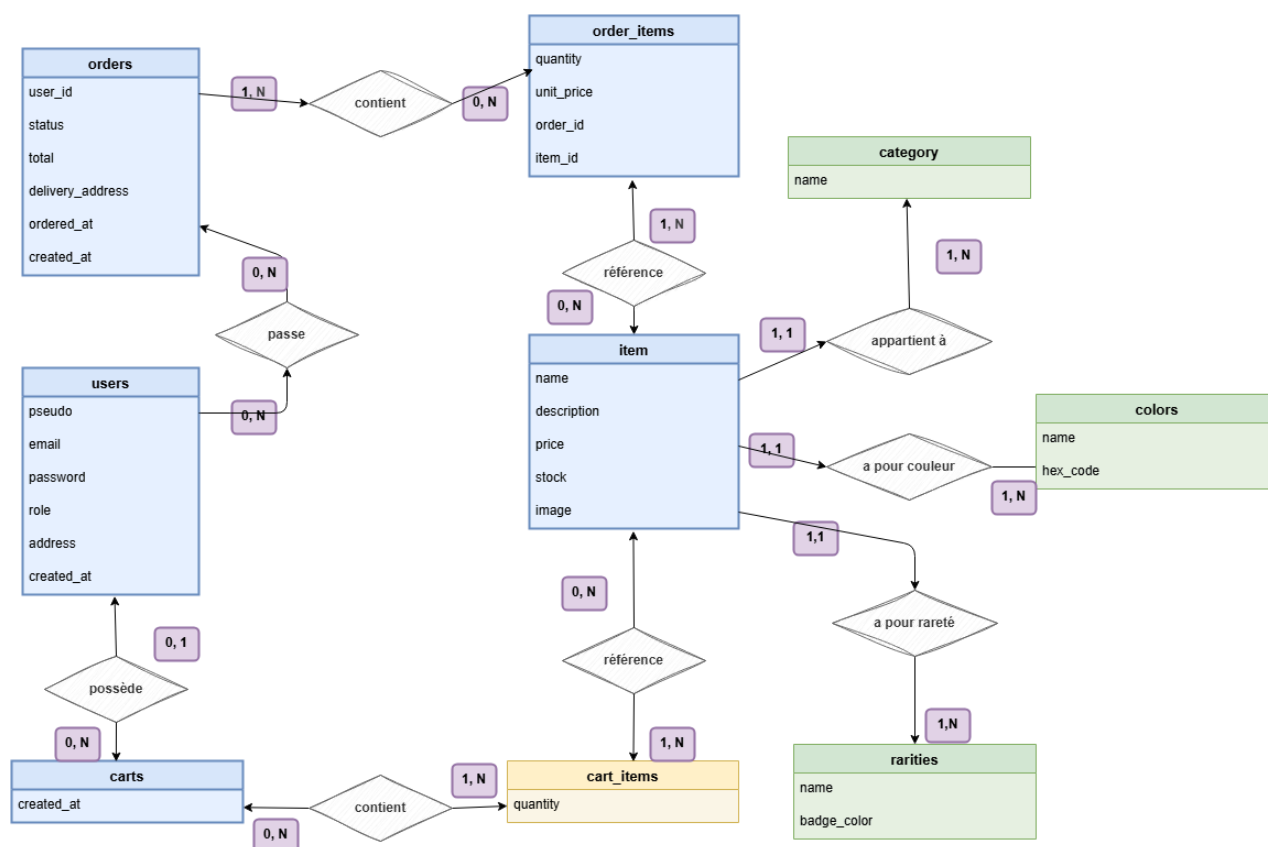
Le RGAA (Référentiel Général d'Amélioration de l'Accessibilité) définit un ensemble de critères pour que les sites web soient utilisables par tous, y compris les personnes en situation de handicap. Je vise un niveau de conformité AA sur les critères les plus importants : navigation au clavier complète, contrastes suffisants entre texte et fond, alternatives textuelles pour toutes les images, structure HTML sémantique rigoureuse, et formulaires correctement étiquetés.

5. Base de données

5.1 MCD – Modèle Conceptuel de Données

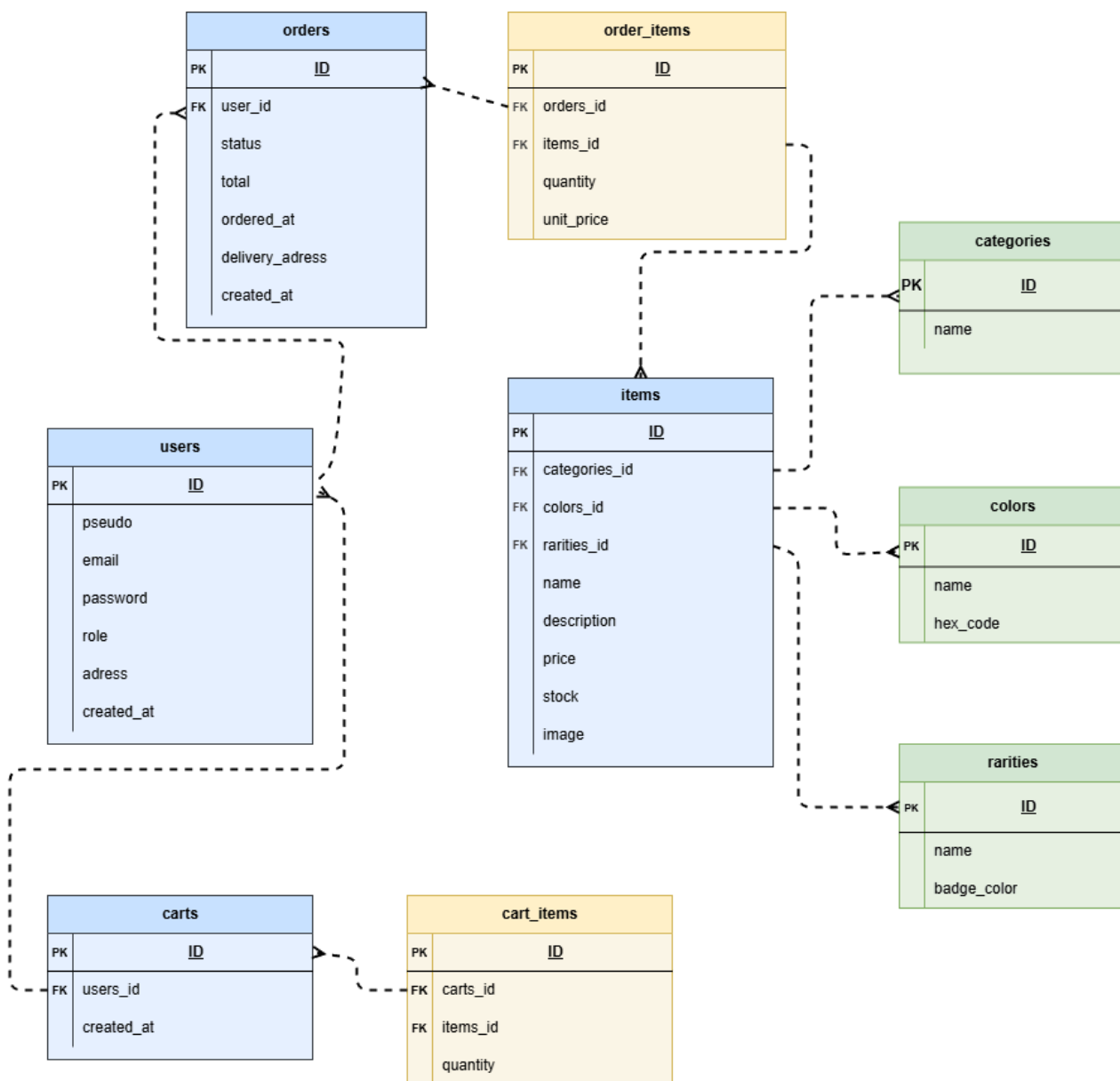
Le MCD est la première étape de la conception de la base de données. On y représente les entités principales du système et leurs relations, sans se soucier encore de l'implémentation technique. Les grandes entités sont : Utilisateur, Produit, Commande, Item, Catégorie, Rareté et Couleur.

Un utilisateur peut passer plusieurs commandes ; une commande contient plusieurs produits ; un produit appartient à une catégorie, à une rareté et une couleur.



5.2 MLD – Modèle Logique de Données

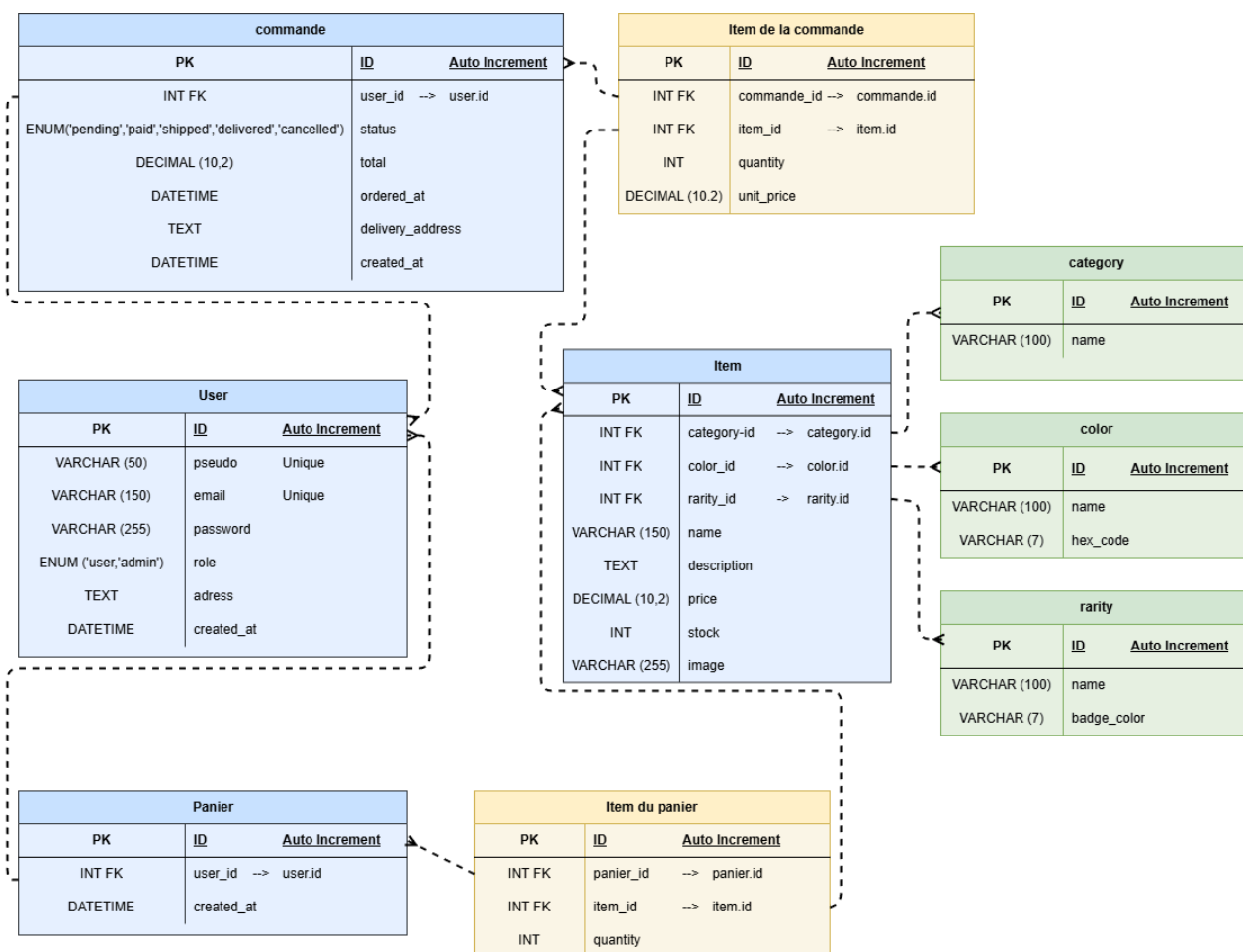
Le MLD est la traduction du MCD en tables relationnelles. On y définit les clés primaires, les clés étrangères, et les cardinalités. La table item sera par exemple liée à la table categories via un champ categorie_id, à la table raretés via rareté_id, et à la table couleurs via couleur_id.



La relation entre commandes et item est une relation plusieurs-à-plusieurs, qui sera matérialisée par une table de liaison commande_produits.

5.3 MPD – Modèle Physique de Données

Le MPD correspond au script SQL de création de la base. C'est ici qu'on définit les types de colonnes précis (VARCHAR, INT, DECIMAL, ENUM...), les contraintes d'intégrité (NOT NULL,



UNIQUE, FOREIGN KEY), et les index utiles pour les performances (notamment sur les colonnes fréquemment filtrées comme rareté_id ou couleur_id).

6. Développement Front-end

6.1 Responsive

Le Responsive Web Design est une approche de conception d'interfaces qui permet à un site de changer de mise en page en fonction de la taille de l'appareil.

Il garantit une lisibilité optimale et une navigation fluide sur smartphone, tablette ou ordinateur. Conçu avec une approche mobile Mobile-First, l'affichage s'ajuste automatiquement grâce aux media queries gérées en Sass.

```
1  $breakpoint-tablet: 768px;
2  $breakpoint-desktop: 1024px;
3
4  @mixin media-up($breakpoint) {
5      @media (min-width: $breakpoint) {
6          @content;
7      }
8  }
9
```

```
.gallery-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;

  @include media-up($breakpoint-tablet) {
    grid-template-columns: repeat(2, 1fr);
  }

  @include media-up($br
```

6.2 Sass

Sass est un préprocesseur qui enrichit le CSS classique avec des fonctionnalités de programmation avancées. Il permet d'utiliser des variables, des mixins et des opérations mathématiques pour rendre le code des styles réutilisable et facile à maintenir.

Grâce à l'imbrication, la structure des feuilles de style devient beaucoup plus propre, lisible et calquée sur le HTML du site. Ce code est ensuite compilé en CSS standard pour assurer une compatibilité totale avec tous les navigateurs.

```
$color-brand: #3498db;
$spacing-base: 8px;
$radius-main: 6px;

@mixin flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```
.btn-primary {
  @include flex-center; // Appel du mixin
  background-color: $color-brand;
  padding: $spacing-base ($spacing-base * 2);
  border-radius: $radius-main;
  color: #ffffff;

  &:hover {
    background-color: darken($color-brand, 10%);
    cursor: pointer;
  }
}
```

6.3 Fetch API

Fetch API est une interface JavaScript qui permet d'effectuer des requêtes réseau asynchrones pour échanger des données avec un serveur. Elle est utilisée pour récupérer ou envoyer des informations (souvent au format JSON) sans avoir à recharger la page web. Grâce à l'utilisation des Promises (**async/await**), elle est simple, lisible et performante pour gérer les réponses et les erreurs. C'est l'outil indispensable pour dynamiser l'affichage de la boutique Rocket League en temps réel.

```
1  async function fetchItems() {
2      const apiUrl = '/api/products.php';
3
4      try {
5          const response = await fetch(apiUrl);
6
7          if (!response.ok) {
8              throw new Error(`Erreur HTTP ! Statut : ${response.status}`);
9          }
10
11         const items = await response.json();
12         console.log("Données chargées avec succès :", items);
13
14     } catch (error) {
15         console.error("Impossible de récupérer les items :", error.message);
16     }
17
18     fetchItems();
19
20
21
22
```

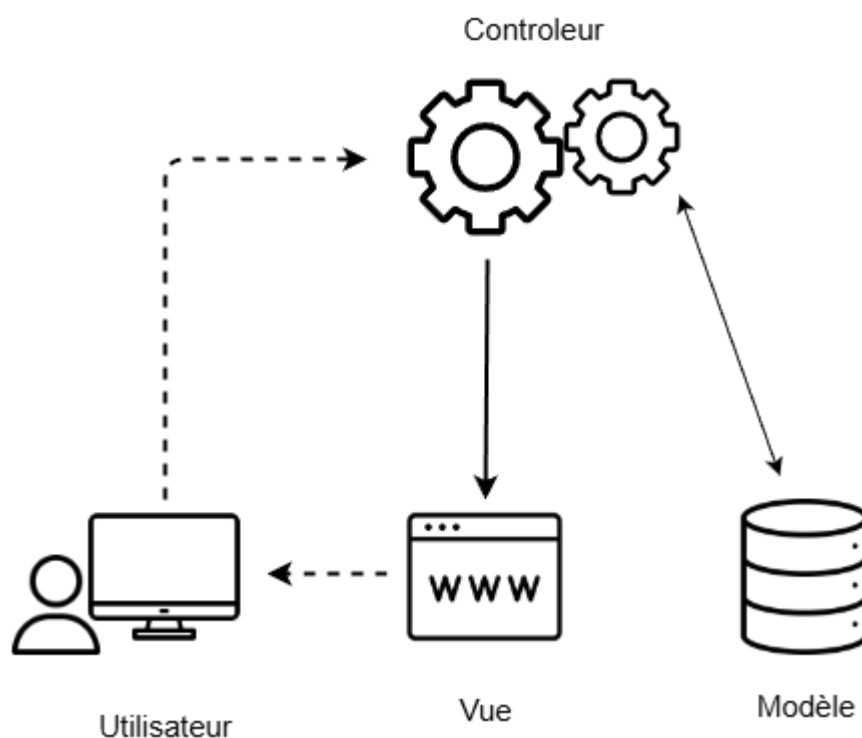
7. Développement Back-end

7.1 Stack technique

- HTML
- SASS
- PHP
- SQL

7.2 Architecture MVC

Le pattern MVC (Modèle-Vue-Contrôleur) est le fil conducteur de toute l'architecture du projet. Chaque couche a une responsabilité unique et bien définie. **Le Modèle** gère tout ce qui touche à la base de données, les requêtes, la validation, la logique métier. **La Vue** s'occupe uniquement de l'affichage : elle reçoit des données et génère le HTML correspondant. **Le Contrôleur** fait le lien entre les deux : il reçoit les requêtes HTTP, appelle les bons modèles, et transmet les données aux bonnes vues.



7.3 API REST

La communication entre le front-end et le back-end passe par une API REST. Les endpoints sont définis selon les ressources principales : produits, catégories, utilisateurs, panier, commandes. Chaque endpoint répond à des méthodes HTTP précises (GET pour lire, POST pour créer, PUT pour modifier, DELETE pour supprimer) et renvoie des données au format JSON.

Cette approche est plus propre qu'un mélange de PHP/HTML qui génère tout en même temps. Ça permet aussi d'envisager plus tard un front-end en React ou Vue.js sans avoir à tout réécrire côté serveur.

7.4 POO / PDO

POO : Le code est structuré en classes (comme "ItemModel" qui hérite d'une classe parente "Model"). Cette approche permet de réutiliser le code commun et de structurer proprement l'architecture MVC.

8. Sécurité

8.1 Authentification & hachage

La sécurité des mots de passe, c'est une des premières choses qu'on apprend à ne jamais négliger. Toute l'authentification repose sur l'algorithme bcrypt, via les fonctions natives PHP `password_hash()` et `password_verify()`. Bcrypt a la particularité d'être volontairement lent à calculer, ce qui le rend résistant aux attaques par force brute. Même si quelqu'un récupérerait la base de données, il n'obtiendrait que des hashes impossibles à inverser directement.

8.2 Protections XSS / Injections SQL

Les injections SQL et les failles XSS sont les deux grands types de failles à connaître quand on développe un site avec des formulaires et une base de données. Les premières permettent à un attaquant d'injecter du code SQL dans une requête pour manipuler la base de données. Les secondes permettent d'injecter du JavaScript malveillant dans des pages vues par d'autres utilisateurs.

La protection contre les injections SQL repose sur l'utilisation systématique de requêtes préparées via PDO : aucune donnée utilisateur n'est jamais insérée directement dans une requête SQL. La protection XSS repose sur la fonction `htmlspecialchars()` appliquée à toutes les données dynamiques affichées dans les vues.

8.3 Conformité RGPD

Le RGPD (Règlement Général sur la Protection des Données) impose plusieurs obligations concrètes. Premièrement, la collecte de données doit être limitée au strict nécessaire : on ne demande pas de numéro de téléphone si on n'en a pas besoin. Deuxièmement, les utilisateurs doivent être informés et consentir à l'utilisation de leurs données, notamment pour les cookies. Troisièmement, tout utilisateur a le droit de demander la suppression de son compte et de toutes ses données personnelles associées — c'est ce qu'on appelle le droit à l'oubli.

Une politique de confidentialité claire sera accessible depuis le pied de page de toutes les pages du site. Elle détaillera les données collectées, la durée de conservation, et les droits de l'utilisateur.

9. Cadre pédagogique

9.1 Blocs de compétences — RNCP 37273

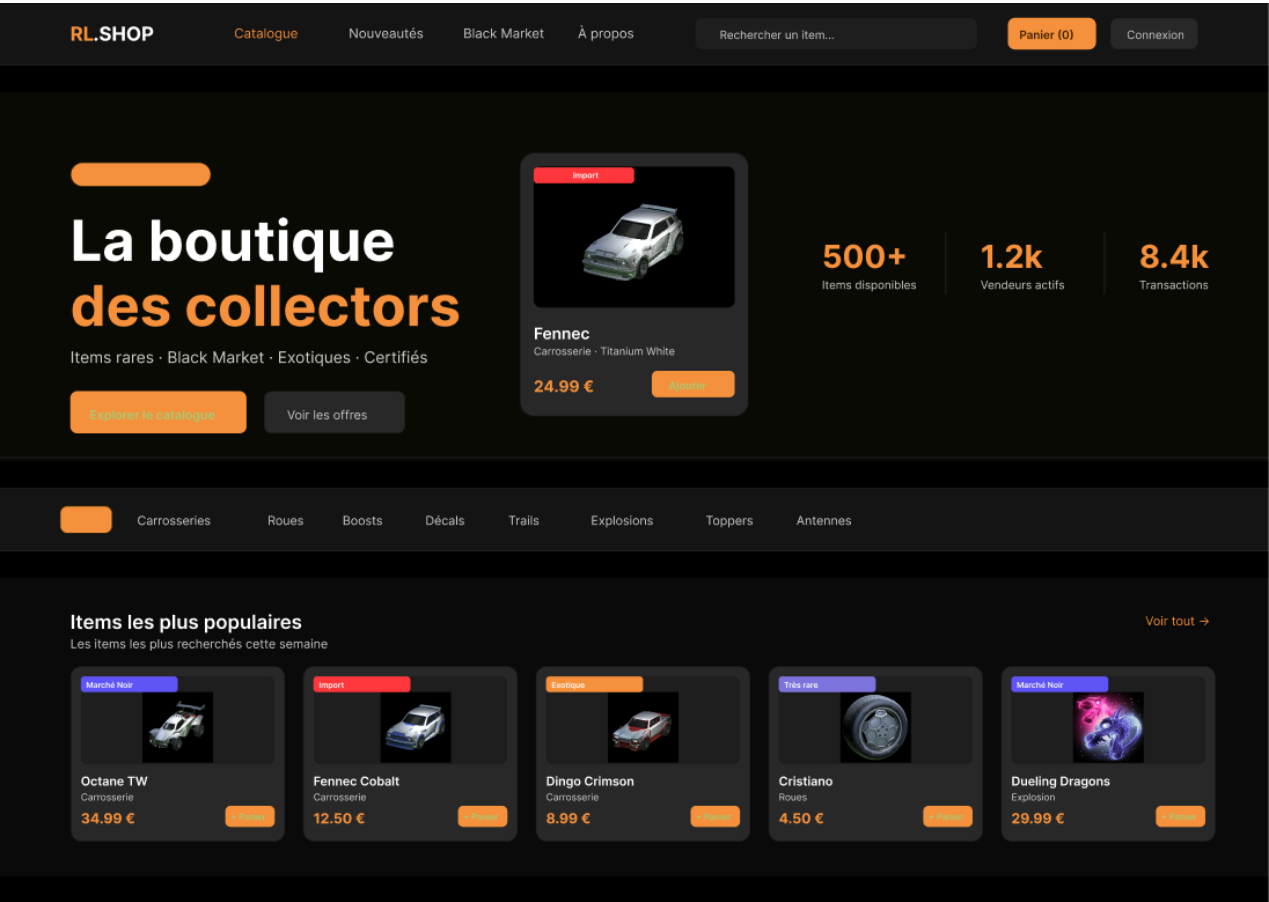
Ce projet est développé dans le cadre du titre professionnel RNCP 37273 — Développeur Web & Web Mobile Fullstack. Il est conçu pour couvrir les blocs 1, 2 et 3 du référentiel de certification, et constitue le support principal de la soutenance.

Le bloc 1 (développement front-end) sera démontré à travers la conception d'une interface mobile-first, responsive et accessible, avec des interactions dynamiques réalisées en JavaScript vanilla. Le bloc 2 (développement back-end) sera illustré par l'architecture MVC en PHP/POO, la gestion de la base de données MySQL, la mise en place de l'API REST et la sécurisation de l'ensemble. Le bloc 3 (gestion de projet et communication) sera couvert par ce cahier des charges, le planning, la documentation technique, et la présentation orale.

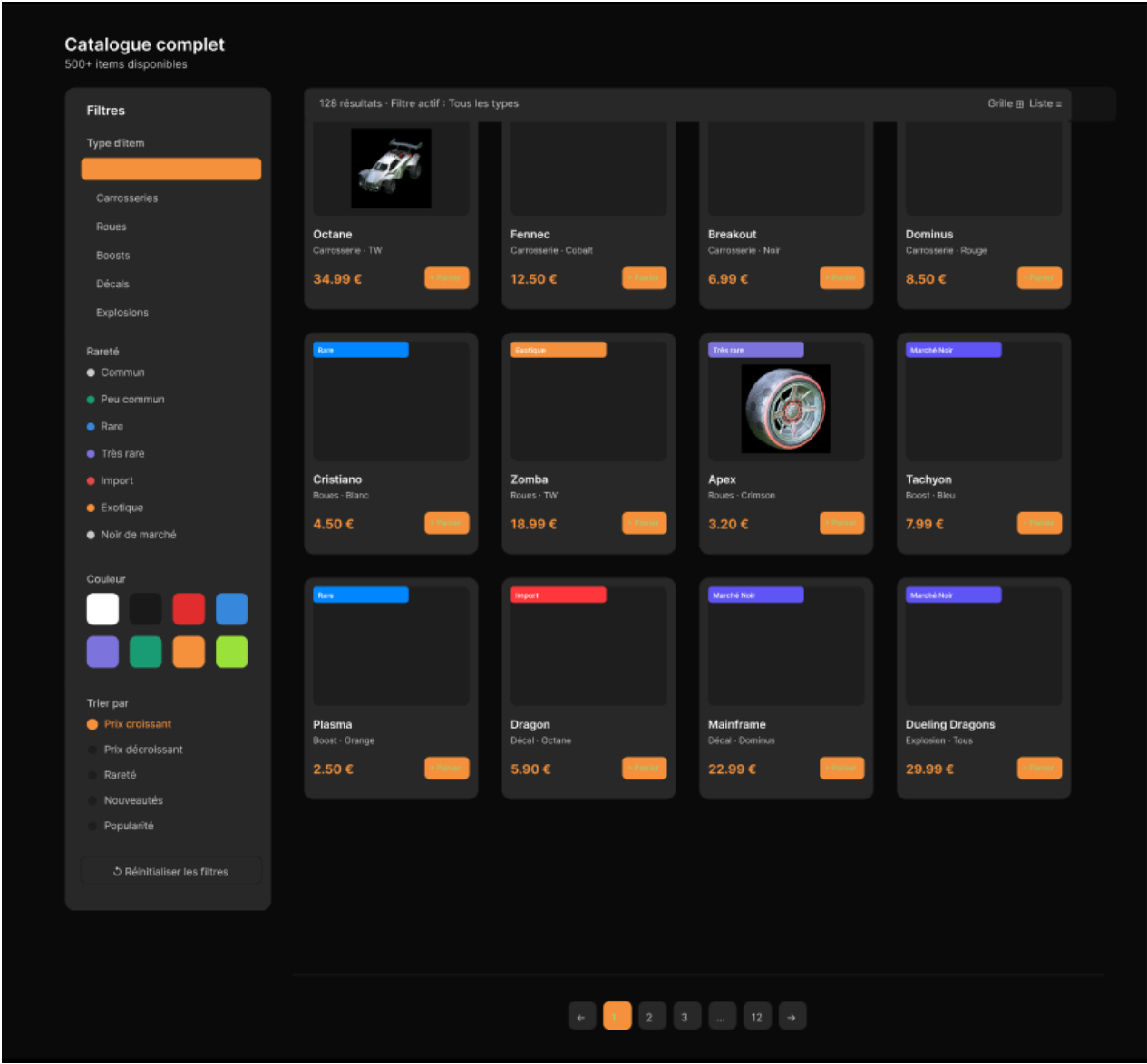
10. Annexe

10.1 Maquette

10.1.1 Header & Hero Section



9.1.2 Main Section



9.1.3 Footer

